Software Architecture & Design (SAD) Specification

Web-Based ML Model Evaluator

IEEE SAD template-aligned architecture document

Project name: Web-Based ML Model Evaluator **Document:** Software Architecture Description (SAD) **Version:** 1.1 - Final as-built architecture **Date:** 4 November 2026 **Prepared by:** Web-Based ML Model Evaluator Project Team

Document control

This SAD follows the supplied IEEE SAD Specification Template and the applicable architecture items in `02TheCheckList.pdf`. It describes the as-built project architecture and identifies implementation and deployment boundaries explicitly. The component diagram source, `ml_evaluator_component_diagram_simple.png`, is included beside this document and embedded in the PDF. Logical components in the diagram are implemented as modules within one Flask application; they are not separate deployed microservices.

Version	Date	Description
1.0	7 Nov 2026	Initial template-aligned, implementation-based SAD.
1.1	4 Nov 2026	Final template-aligned SAD documenting the project architecture, traceability, interfaces, security design, and verification boundaries.

1. Introduction

1.1 Purpose

This document gives developers, reviewers, and maintainers a consistent view of the system's architecture, design decisions, interfaces, security boundaries, important runtime interactions, and relationship to the approved requirements. It is intended to support implementation review, verification, maintenance, and project demonstrations.

1.2 Scope

The Web-Based ML Model Evaluator is a browser-based application for account access, CSV dataset upload and inspection, preprocessing, supervised model training and evaluation, run history and comparison, artifact/report downloads, Instructor review of shared results, and Admin account/dataset/audit management. The system runs as a Flask modular monolith: Flask serves the web UI and JSON API, SQLAlchemy persists metadata in SQLite by default, files are held in local configured storage, and scikit-learn performs model processing in the backend process.

This SAD is an as-built description, not a proposal for future capabilities. It does not claim independently deployed services, production hosting, public TLS verification, encryption at rest, high availability, or completed production penetration testing.

1.3 Definitions, acronyms, and abbreviations

Term	Meaning
API	Application Programming Interface; here, HTTP endpoints returning JSON or files.
SAD	Software Architecture Description.
SRS	Software Requirements Specification.
Student	Standard account role; owns datasets, models, and evaluations.
Instructor	Read-only role for results explicitly shared by Students.
Admin	Privileged role for user management, dataset oversight, and activity-log access.
ML	Machine Learning.
ORM	Object-Relational Mapper; SQLAlchemy maps application entities to relational tables.
Modular monolith	One deployable application organized into internal modules, rather than separately deployed services.
Artifact	Uploaded dataset or serialized trained model stored on the filesystem.

1.4 References

1. IEEE Std 1471-2000, *Recommended Practice for Architectural Description of Software-Intensive Systems* (standard cited on the supplied SAD template).
2. IEEE Std 1016-2009, *IEEE Standard for Information Technology - Systems Design - Software Design Descriptions* (reference cited by the supplied template).
3. ISO/IEC/IEEE 42010:2011, *Systems and software engineering - Architecture description* (reference cited by the supplied template).
4. Project SRS, `Corrected_SRS_v1.2.pdf`; baseline `Corrected_SRS.pdf`.
5. Project Security Requirements, `Corrected_Security_Requirements.pdf`.
6. Project Test Plan and execution record, `Corrected_Test_Plan_Executed.pdf` and `Corrected_Test_Plan_Execution_Results.md`.
7. Project component diagram, `ml_evaluator_component_diagram_simple.png`.
8. Project database definition, `Database_Schema.md` and `Database_Schema.pdf`.
9. Project use-case flows, `ML_Evaluator_Use_Case_Flows.pdf`.
10. Repository source: `backend/app.py`, `backend/config.py`, `backend/routes/`, `backend/models/`, `backend/ml/`, `frontend/`.

2. Architectural Representation and Patterns

2.1 Architectural pattern/style and requirements alignment

The implementation follows a **layered modular-monolith style** with a browser presentation layer, Flask HTTP/API boundary, application route and authorization modules, ML/data-processing modules, relational persistence, and filesystem artifact storage. The browser calls the same-origin Flask API using JSON and multipart form requests. The backend coordinates validation, access checks, processing, and persistence. The ML pipeline executes in-process; there is no remote ML service or job queue.

Architectural concern	Design choice	Requirements supported
Clear browser/backend boundary	Static HTML/CSS/JavaScript client calls JSON REST routes.	FR-6.2, NFR-8, NFR-10
Resource privacy	Backend derives identity from the validated session and scopes Student resources by owner.	FR-1.4, FR-7.5, NFR-3 to NFR-5
Role-based workflows	Server-side Student, Instructor, and Admin authorization. Instructor reads only shared results; Admin routes are separately protected.	FR-7.1 to FR-7.5
Maintainable feature boundaries	Flask blueprints, SQLAlchemy models, and distinct preprocessing/training/evaluation modules.	NFR-10
Local reproducibility	SQLite and local artifact storage are defaults; runtime locations and database can be configured.	FR-4.3, NFR-7, NFR-14
Bounded input and abuse	CSV-only validation, 16 MiB request limit, login throttles, and configured sensitive-route limits.	FR-2.2, NFR-11, security requirements

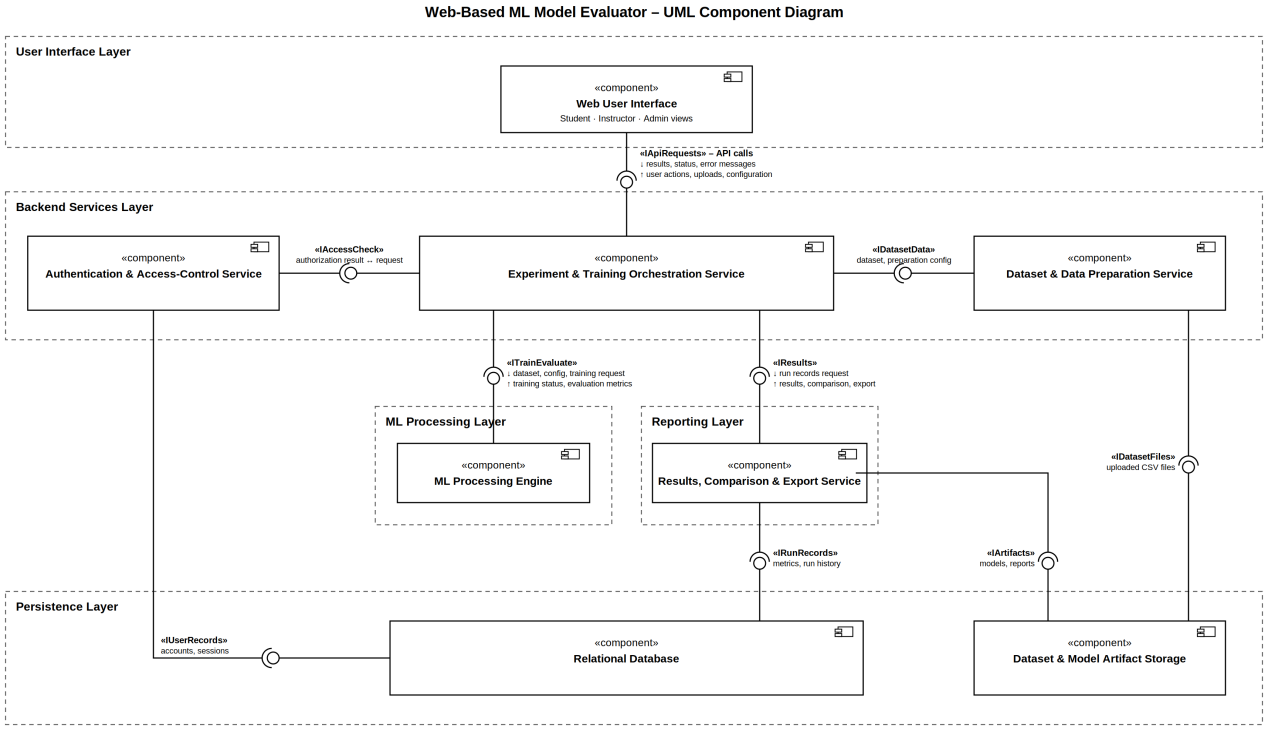
2.2 Runtime and deployment view

The default local setup is one Python process bound to `127.0.0.1:5000`. Flask serves `frontend/` and `/api/*`. SQLite defaults to `backend/ml_evaluator.db`; uploads and serialized models are stored in configurable directories under the backend. A deployment must provide a stable `SECRET_KEY`, configure secure cookies when using HTTPS, use a production WSGI server, and provide persistent database/artifact storage. TLS termination is an environment/reverse-proxy responsibility. Horizontal scaling would require shared persistence and coordinated session/artifact handling.

3. System Architecture and High-Level Design

3.1 Component diagram and descriptions

The following is the project component diagram supplied with this SAD source. It shows logical responsibilities and data flows. The named services map to internal Flask modules in this implementation, not to separate network services.



Design component ID and name	Implementation mapping	Description and mapped SRS requirements
COMP-01 - Web User Interface (Student, Instructor, Admin views)	`frontend/login.html`, `frontend/index.html`, `frontend/js/app.js`, `frontend/css/style.css`	Collects input and renders role-aware workflows, validation feedback, datasets, metrics, and charts. SRS: FR-1.1 to FR-7.5; NFR-1, NFR-2, NFR-8, NFR-10, NFR-14.
COMP-02 - Authentication and Access-Control Service	`backend/routes/auth_routes.py`, `backend/routes/auth_utils.py`, `backend/models/user_session.py`	Registers and authenticates accounts, creates/revokes sessions, enforces roles and owner checks, applies throttles, and records security activity. SRS: FR-1.1 to FR-1.4, FR-7.5; NFR-3 to NFR-5; security requirements.
COMP-03 - Dataset and Data Preparation Service	`backend/routes/dataset_routes.py`, `backend/ml/preprocessor.py`	Validates and stores CSVs; returns metadata, preview, and statistics; replaces owned datasets and prepares features. SRS: FR-2.1 to FR-2.5, FR-3.1; NFR-11, NFR-12.
COMP-04 - Experiment and Training Orchestration Service	`backend/routes/ml_routes.py`	Validates training/evaluation requests, resolves owned datasets, coordinates ML steps, persists run metadata, and serves history/comparison/download operations. SRS: FR-3.2, FR-3.3, FR-4.1 to FR-4.3, FR-6.1.
COMP-05 - ML Processing Engine	`backend/ml/trainer.py`, `backend/ml/evaluator.py`	Builds supported scikit-learn pipelines, trains models, computes classification/regression metrics, and produces evaluation outputs synchronously in Flask. SRS: FR-4.1 to FR-4.3, FR-5.1; NFR-1, NFR-2, NFR-6.
COMP-06 - Results, Comparison and Export Service	`backend/routes/ml_routes.py`, `backend/models/evaluation.py`	Persists and returns results, compares compatible runs, controls Instructor sharing, and serves authorized model/report downloads. SRS: FR-5.1 to FR-5.4, FR-6.1, FR-7.4.

Design component ID and name	Implementation mapping	Description and mapped SRS requirements
COMP-07 - Relational Database	`backend/database/db.py`, `backend/models/`; SQLite by default	Stores users, datasets, model/run metadata, evaluations, sessions, activity events, and login-throttle records. SRS: FR-4.3, FR-6.1; NFR-6, NFR-7, NFR-12.
COMP-08 - Dataset and Model Artifact Storage	Configured upload and saved-model directories	Stores CSV and serialized model files; database rows retain ownership, metadata, and artifact paths, not file contents. SRS: FR-2.1 to FR-2.5, FR-4.2, FR-5.4; NFR-6, NFR-7, NFR-12.
COMP-09 - Admin Service	`backend/routes/admin_routes.py`	Provides protected user management, dataset oversight, and activity-log reads. SRS: FR-7.1 to FR-7.3.

3.1.1 Data and persistence view

The relational schema has seven tables: `users`, `datasets`, `models`, `evaluation_results`, `user_sessions`, `activity_logs`, and `login_attempts`. One user may own many datasets, models, evaluations, sessions, and activity records. Models refer to their source dataset. An evaluation refers to a model and owner; database integrity controls ensure that the evaluation owner matches the model owner. Uploaded CSVs and model binaries remain filesystem artifacts. Backups must capture the database and the configured upload/model directories together. The complete field and relationship description is in `Database_Schema.md`.

3.2 Security architecture

3.2.1 Identity and session management

Passwords are stored as Werkzeug password hashes. Successful login creates a signed opaque browser cookie and a server-side `user_sessions` record. Protected requests validate the session record, user state, revocation state, and 24-hour idle lifetime. Logout revokes the server-side record and clears the cookie. Cookies are HttpOnly and SameSite=Lax; Secure is enabled through `SESSION_COOKIE_SECURE=true` for HTTPS deployments. The random development secret default is not suitable for multi-process or production use; deployments must set a stable secret.

3.2.2 Authorization and data isolation

Authorization is enforced in backend route decorators and resource queries, not solely by frontend visibility. Student dataset/model/evaluation operations are owner-scoped. Instructors receive read-only access to explicitly shared evaluations. Admin routes require the Admin role. New registrations receive Student role; callers cannot self-assign privileged roles. Initial Admin setup uses the trusted local `flask create-admin` command, after which Admin account management is available in the application.

3.2.3 Input, abuse, and audit controls

Uploads accept CSV only and are bounded by a 16 MiB total request-body limit by default; minimum data shape is two rows and two columns. Login throttling defaults to five attempts per normalized account and 20 per source IP within a rolling 15-minute window. Configured per-IP limits also cover selected registration, dataset mutation, training, comparison, download, sharing, and Admin endpoints. Throttle identifiers use keyed digests. Security-relevant events are written to activity logs with secrets excluded.

3.2.4 Transport and operational security boundaries

The application restricts configured local cross-origin access and uses same-origin requests by default. The Flask app can issue Secure cookies but does not itself provide public certificate issuance or TLS termination. At-rest encryption is not implemented by the application. The local database and files depend on operating-system and deployment storage protection. Dependency audit, production TLS, backup restore, and deployment evidence are tracked in the test/release documents; local tests are not a production security certification.

3.3 Requirements traceability matrix

This table links SRS requirement groups to architecture elements and implementation evidence. It is a design trace, not a claim that every requirement has been accepted in a deployed environment. Requirement numbering follows `Corrected_SRS_v1.2.pdf`; the exact title/acceptance criteria remain authoritative in that SRS.

SRS requirement	Architecture elements	Implementation evidence
FR-1.1 to FR-1.4 - registration, login, logout, protected access	Auth routes, session model/helpers, login UI	`backend/routes/auth_routes.py`, `backend/routes/auth_utils.py`, `backend/models/user_session.py`
FR-2.1 to FR-2.5 - upload, validation, preview, statistics, replacement	Dataset routes, Dataset model, upload storage	`backend/routes/dataset_routes.py`, `backend/models/dataset.py`
FR-3.1 to FR-3.3 - preprocessing, target selection, train/test split	Preprocessor, ML routes, training UI	`backend/ml/preprocessor.py`, `backend/routes/ml_routes.py`, `frontend/`
FR-4.1 to FR-4.3 - algorithm selection, training, run/configuration record	Algorithm catalog, trainer, model/run persistence	`backend/ml/trainer.py`, `backend/routes/ml_routes.py`, `backend/models/model.py`
FR-5.1 to FR-5.4 - metrics, visualization, comparison, downloads	Evaluator, evaluation persistence, result UI and artifact routes	`backend/ml/evaluator.py`, `backend/routes/ml_routes.py`, `frontend/js/app.js`
FR-6.1 to FR-6.3 - history, API behavior, errors	ML routes, JSON API, frontend client	`backend/routes/ml_routes.py`, `backend/app.py`, `frontend/js/app.js`
FR-7.1 to FR-7.5 - Admin controls, Instructor shared results, role enforcement	Auth helpers, Admin routes, Instructor result routes	`backend/routes/auth_utils.py`, `backend/routes/admin_routes.py`, `backend/routes/ml_routes.py`
NFR-1 to NFR-2 - responsiveness and user-visible operation feedback	API handlers, synchronous ML calls, browser status UI	`backend/routes/`, `frontend/js/app.js`; measured latency evidence in execution record
NFR-3 to NFR-5 - credential security, authorization, privacy	Password hashing, sessions, RBAC, owner-scoped queries	`backend/routes/auth_*`, `backend/models/user_session.py`, dataset/model/evaluation queries
NFR-6 to NFR-7 - reliability and reproducibility	Relational records, saved configuration, artifact association	`backend/models/`, `backend/ml/`, `documents/Backup_and_Restore.md`
NFR-8 to NFR-10 - usability, browser compatibility, maintainability	Static frontend, REST boundary, modular backend	`frontend/`, `backend/routes/`, `backend/ml/`; browser evidence in execution record
NFR-11 to NFR-14 - resource constraints, artifact/metadata linkage, safe errors, setup/deployment	Request limits, relational references, JSON error boundary, setup/configuration	`backend/config.py`, `backend/app.py`, `backend/models/`, `README.md`
Security requirements - sessions, role protection, rate limiting, audit, backup, transport	Auth/session, access helpers, audit records, filesystem/database operations, deployment boundary	`backend/routes/`, `backend/config.py`, `documents/Corrected_Security_Requirements.pdf`, `documents/Backup_and_Restore.md`

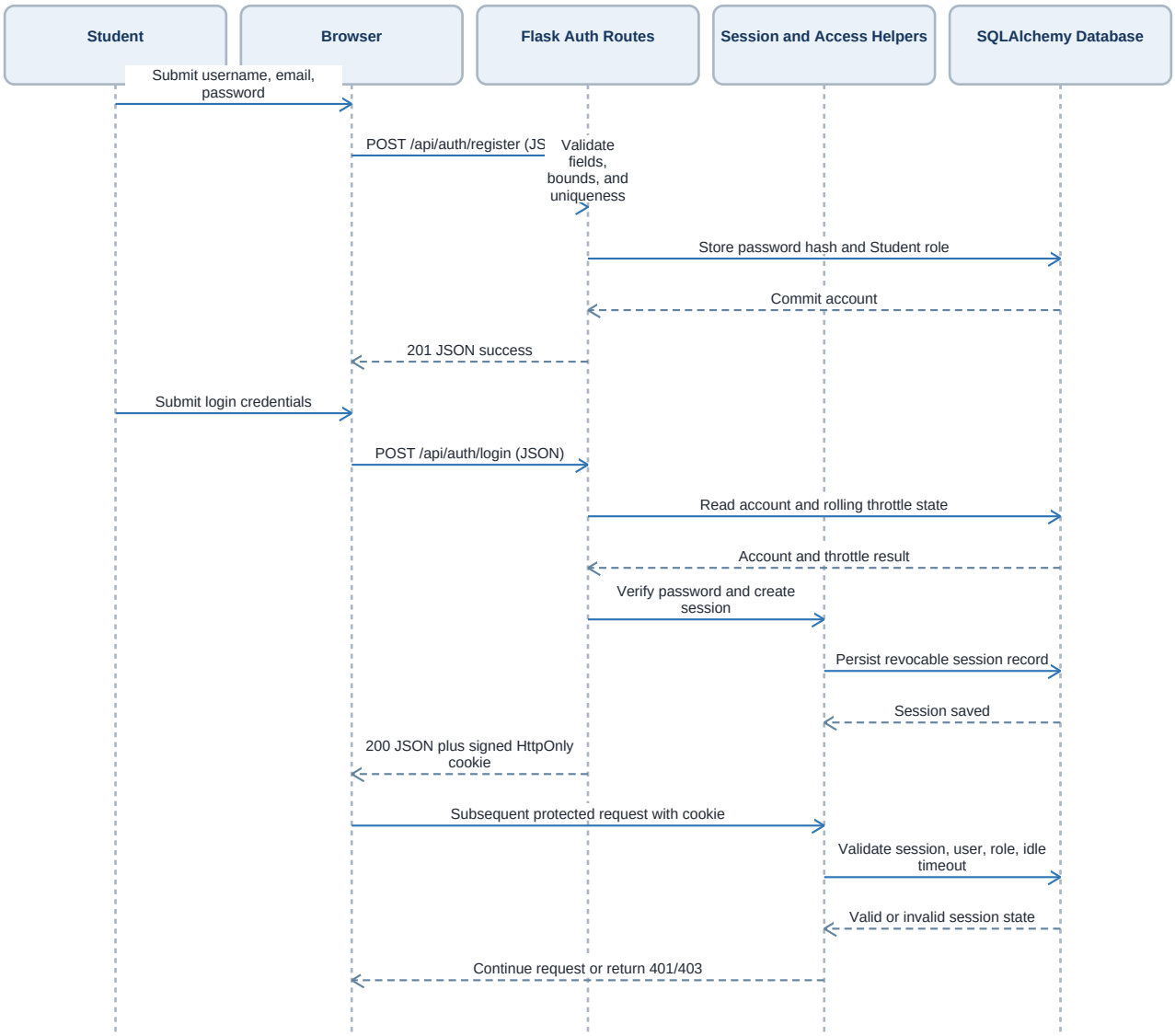
4. Detailed Design and Interface Protocols

4.1 UML sequence diagrams

The two sequences below describe the implemented request interactions. They are sequence diagrams included in this SAD; the existing use-case and component diagrams remain unchanged.

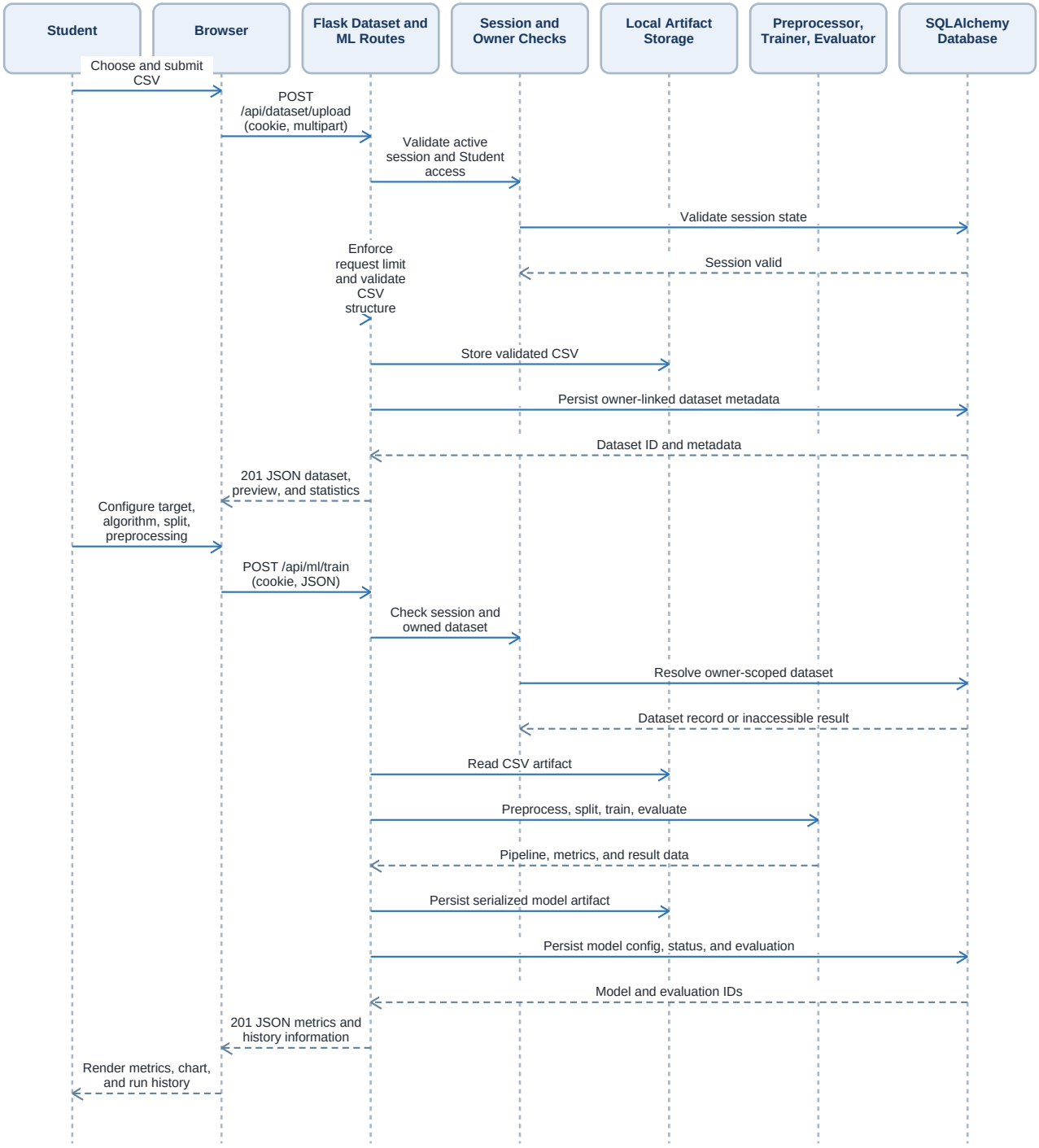
4.1.1 Authentication workflow

Registration validates the required fields and creates a Student account. At login, the authentication routes check the account/IP throttle before verifying the password. A successful login creates a server-side revocable session and returns a signed cookie. Protected requests validate the session and user state before role and ownership checks; invalid sessions and disallowed roles receive 401 and 403 responses respectively.


Student
Browser
Flask Auth Routes
Session and Access Helpers
SQLAlchemy Database
Submit username, email, password
POST /api/auth/register (JS
Validate fields, bounds, and uniqueness
Store password hash and Student role
Commit account
201 JSON success
Submit login credentials
POST /api/auth/login (JSON)
Read account and rolling throttle state
Account and throttle result
Verify password and create session
Persist revocable session record
Session saved
200 JSON plus signed HttpOnly cookie
Subsequent protected request with cookie
Validate session, user, role, idle timeout
Valid or invalid session state
Continue request or return 401/403

4.1.2 Core data transaction: upload, train, evaluate

The dataset route validates the authenticated Student, request-size limit, CSV type, and tabular structure before storing the file and owner-linked metadata. The training route verifies the dataset belongs to that user, applies configured preprocessing and train/test splitting, trains and evaluates the selected compatible estimator, then stores the model artifact and result metadata. It returns the persisted identifiers, split details, and metrics for the browser to display.



4.2 API and interface design

4.2.1 Protocol conventions

The web API uses HTTP under `/api`. JSON request/response bodies use UTF-8; file uploads use `multipart/form-data`; browser authentication uses the signed session cookie. Read operations use GET, state-changing operations use POST or PATCH, and supported deletion uses DELETE. Successful reads and updates normally return 200, created records return 201, and downloads return file responses. API errors use a JSON `error` message and the applicable 4xx/5xx status. Authentication, role checks, and ownership are enforced on the server. The endpoint catalog lists each route's inputs/purpose and access control; Section 4.3 summarizes error status behavior.

4.2.2 Endpoint catalog

Method and path	Input / purpose	Access
`POST /api/auth/register`	JSON `username`, `email`, `password`; creates a Student account.	Public, rate-limited
`POST /api/auth/login`	JSON `username`, `password`; establishes session cookie.	Public, throttled by account and IP
`GET /api/auth/session`	Returns current authenticated identity/role.	Session required
`POST /api/auth/logout`	Revokes current server-side session and clears cookie.	Session required
`GET /api/health`	Returns basic service health/version JSON.	Public
`GET /api/dataset`	Lists current user's datasets.	Authenticated, owner-scoped
`GET /api/dataset/{dataset_id}`	Reads owned dataset metadata.	Authenticated, owner-scoped
`POST /api/dataset/upload`	Multipart CSV file; validates and stores dataset.	Authenticated, Student access
`POST /api/dataset/{dataset_id}/replace`	Multipart replacement CSV.	Authenticated, owner-scoped
`GET /api/dataset/{dataset_id}/preview`	Returns preview rows and columns.	Authenticated, owner-scoped
`GET /api/dataset/{dataset_id}/statistics`	Returns dimensions and descriptive/missing-value statistics.	Authenticated, owner-scoped
`POST /api/dataset/{dataset_id}/preprocess`	JSON preprocessing choices; returns prepared-data summary.	Authenticated, owner-scoped
`GET /api/ml/algorithms`	Lists supported algorithms and task compatibility.	Authenticated
`GET /api/ml/models`, `GET /api/ml/models/{model_id}`	Lists or reads owned model metadata.	Authenticated, owner-scoped
`GET /api/ml/evaluations`, `GET /api/ml/evaluations/{evaluation_id}`	Lists or reads owned evaluations.	Authenticated, owner-scoped
`POST /api/ml/train`	JSON dataset/target/task/algorithm/split/preprocessing configuration; trains and records run.	Authenticated, Student access; rate-limited
`POST /api/ml/evaluate`	JSON model/evaluation configuration; evaluates an authorized model.	Authenticated, owner-scoped
`POST /api/ml/compare`	JSON evaluation identifiers; compares compatible owned runs.	Authenticated, owner-scoped
`GET /api/ml/models/{model_id}/download`	Downloads serialized model artifact.	Authenticated, owner-scoped
`GET /api/ml/evaluations/{evaluation_id}/report`	Downloads evaluation report.	Authenticated, owner-scoped
`PATCH /api/ml/evaluations/{evaluation_id}/sharing`	JSON sharing state; owner controls Instructor visibility.	Authenticated, evaluation owner

Method and path	Input / purpose	Access
`GET /api/ml/instructor/evaluations`	Lists evaluations shared with Instructors.	Instructor role
`GET /api/ml/instructor/evaluations/{evaluation_id}`	Reads one shared evaluation.	Instructor role; shared result only
`POST /api/ml/instructor/compare`	JSON identifiers for compatible shared results.	Instructor role; shared results only
`GET /api/admin/users`, `GET /api/admin/users/{user_id}`	Lists users or reads account details.	Admin role
`PATCH /api/admin/users/{user_id}`	Updates supported role/activation fields.	Admin role
`GET /api/admin/datasets`, `GET /api/admin/datasets/{dataset_id}`	Lists or inspects dataset metadata and owner.	Admin role
`DELETE /api/admin/datasets/{dataset_id}`	Deletes supported dataset record/artifact according to route behavior.	Admin role
`GET /api/admin/logs`	Reads activity records.	Admin role

4.2.3 Representative request and response schemas

Endpoint	Request schema	Success response and status
`POST /api/auth/register`	JSON object: required string fields `username` (maximum 80 characters), `email` (maximum 120 characters), and `password` (8-128 characters). Role is not accepted from the client.	`201`: `{"message":"Registration successful","user_id":42,"role":"student"}`.
`POST /api/auth/login`	JSON object: required string fields `username` and `password`.	`200`: user identity/role and `redirect_url`; signed session cookie set by the response. Invalid credentials return a generic `401`; throttled requests return `429` with `Retry-After`.
`POST /api/dataset/upload`	`multipart/form-data` with required `file` containing a validated `.csv`; total request body is limited by configuration (default 16 MiB).	`201`: `message`, `dataset_id`, `filename`, row/column counts, column names, and file size.
`POST /api/ml/train`	JSON object: `dataset_id` (integer), `target_column` (string), `task_type` (`classification` or `regression`), `algorithm` (supported catalog value), optional `test_size` (number between 0 and 1), optional integer `random_state`, and optional `preprocessing` object.	`201`: `model_id`, `evaluation_id`, task/algorithm, metrics, confusion matrix or report as applicable, and split details.

The exact algorithm/task combinations are exposed by `GET /api/ml/algorithms`; invalid or incompatible fields return a JSON error. Run comparison accepts 2-10 distinct evaluation IDs and requires compatible dataset, target, task, split, seed, and preprocessing settings. The API catalog and route validators are authoritative for optional fields and endpoint-specific details.

4.3 Error handling protocols

Exception / fault	HTTP status	System reaction and fallback action	Logging strategy
Malformed JSON, missing fields, unsupported values, or invalid CSV	400 or 422 as returned by route	Return JSON validation detail; correct the request and retry.	Expected validation failures are not all audit events; request secrets and file contents are not included in client errors.
Missing, expired, or revoked authentication session	401	Reject protected access; sign in again.	Authentication events are recorded where implemented; passwords and session identifiers are excluded from activity details.
Authenticated account lacks required role	403	Return access denied; do not retry using the same role.	Privileged activity is recorded where implemented; no centralized SIEM integration is claimed.
Owner-scoped resource missing or inaccessible	404	Return the same not-found behavior to avoid disclosing another user's resource.	No user-controlled file paths or secret values are intentionally logged.
Duplicate account identity or conflicting state	409 where handled	Return a conflict response; resolve the duplicate or state conflict.	Route-specific activity logging applies to supported account-management operations.
Request body exceeds configured size	413	Reject the request; reduce the request size below the configured limit.	Flask request-limit handler returns a bounded message; no upload contents are logged.
Configured rate limit exceeded	429 with `Retry-After`	Return retry delay and wait before retrying.	Throttle identifiers use keyed digests; rejected requests do not log credentials or session values.
Unexpected application, database, or artifact failure	Generic 5xx where handled	Return a generic error; no circuit breaker or background retry queue is implemented.	Specific failure paths use server-side exception logging; `ActivityLog` records supported security/admin events with secrets excluded. Logging is not a centralized monitoring service.

The frontend presents API errors as workflow feedback. A generic error must not reveal password hashes, session values, secret keys, filesystem-sensitive details, or stack traces. Database outages are not a separately designed circuit-breaker protocol and may surface as a generic server failure.

5. Architecture Constraints, Risks, and Verification Notes

1. Flask's built-in development server is for local development; a deployment needs a production WSGI server and a TLS-capable edge.
2. The 16 MiB upload limit includes multipart overhead; accepted file bytes may be slightly lower.
3. Training runs synchronously in a web request and can occupy a worker for large workloads. No worker queue, cancellation, or progress-event channel exists in this architecture.
4. SQLite and local filesystem storage describe the default single-instance setup. Scaling across instances requires shared persistence, stable secrets, and an artifact consistency/backup plan.
5. Database records and artifacts must be backed up/restored together.
6. The application does not implement encryption at rest. HTTPS/public TLS and trusted certificate verification depend on deployment configuration.
7. The local execution record provides automated and browser evidence, but does not itself prove a successful hosted CI run or a production deployment.

6. Checklist Coverage

Applicable SAD checklist item	Where addressed
IEEE-style architecture document structure	Sections 1-4 follow the supplied SAD template headings.
Architecture pattern/style and requirements alignment	Section 2.1.
Component diagram and descriptions	Section 3.1, the included component diagram source, and component responsibility/requirement mappings.
Security architecture	Section 3.2.
Requirements traceability	Section 3.3.
At least two sequence diagrams	Sections 4.1.1 and 4.1.2.
API and interface design	Section 4.2.
Error handling protocols	Section 4.3.

The checklist's non-SAD items (such as project-board evidence, sprint demonstration, and test-plan execution) remain documented in their corresponding project records and are not represented as SAD sections.

References

- IEEE Std 1471-2000, IEEE Std 1016-2009, and ISO/IEC/IEEE 42010:2011, as referenced by or relevant to the supplied IEEE SAD Specification Template.
- `Corrected_SRS_v1.2.pdf` - authoritative project requirements and security objectives.
- `Corrected_Security_Requirements.pdf` - security controls and verification expectations.
- `Corrected_Test_Plan_Executed.pdf` and `Corrected_Test_Plan_Execution_Results.md` - planned cases and available execution evidence.
- `ml_evaluator_component_diagram_simple.png` - logical component view included beside the SAD source and embedded in the SAD PDF.
- `Database_Schema.md` - relational entities, relationships, and integrity constraints.
- `ML_Evaluator_Use_Case_Flows.pdf` - actor workflows.
- `Backup_and_Restore.md` - local backup/restore procedure and boundaries.